

Lab 13

Lambda calculus

Goals

In this lab you will learn to:

1. Work with lambda calculus
2. Use the mikrokosmos lambda interpreter

Resources

Table 13.1: Lab Resources

Resource	Link
mikrokosmos - Educational lambda calculus interpreter	https://github.com/mroman42/mikrokosmos
Lambda Calculus: PyCon 2019 Tutorial (Screencast) - David Beazley	https://youtu.be/5C6sv7-eTKg
Lambda Calculus - Computerphile	https://youtu.be/eis11j_iGMs

To review

Skim the lectures on λ -calculus (weeks #12 and #13). Study the following:

- Church numerals (the way natural numbers can be encoded in λ -calculus)
- encoding of logical true and false $\mathcal{T}$ and $\mathcal{F}$ and of logical operations $\wedge, \vee, \neg$, as well as **COND**
- defining expressions for successor **SUCC**, addition **PLUS**, test-if-zero **isZ** and predecessor **PRED**
- how to do a recursive call using the Y combinator
- de Bruijn notation for variables
- combinators **S**, **K** and **I** and proving combinator equivalences (e.g., $\mathbf{I} \equiv \mathbf{S} \mathbf{K} \mathbf{K}$)

13.1 Installing mikrokosmos

All examples of this lab will be run in mikrokosmos¹, a didactic interpreter for λ -calculus.

13.1.1 Using provided binaries (recommended)

Simply download and unzip the executable suitable for your platform (i.e. mikrokosmos-win.zip for Windows and mikrokosmos-linux.zip for Linux).



Note 13.1.1

On Windows, you might need to use the Git Bash terminal to see the colored output of mikrokosmos correctly.

You might also need to change the theme of Git Bash to easily see the colored output. This can be done by right clicking on the title bar of the window (i.e. on the Git logo on the top left) and clicking **Options...**, then selecting a theme like **dracula** or **flat-ui**.

13.1.2 Using the online interpreter

You can run the online interpreter at <https://mroman42.github.io/mikrokosmos>.

13.2 Mikrokosmos tutorial

To start mikrokosmos, just type:

Git Bash session

```
./mikrokosmos.exe
Welcome to the Mikrokosmos Lambda Interpreter!
Version 0.8.0. GNU General Public License Version 3.
```

Definitions for some basic expressions are automatically loaded and made available.

To quit, type at the mikrokosmos prompt:

mikrokosmos session

```
mikro> :quit
```

You may get help by typing:

mikrokosmos session

```
mikro> :help
```

To see the step-by-step reductions, in de Bruijn notation, switch the verbose mode on:

mikrokosmos session

```
mikro> :verbose on
```

¹available at <https://mroman42.github.io/mikrokosmos/tutorial.html>

Example 1: if you want to see the definition of 1 loaded in `mikrokosmos`, type at its prompt:

```
mikrokosmos session
mikro> 1
```

and you should get the following answer:

$\lambda a. \lambda b. a \ b \Rightarrow 1$

If you are in verbose mode, you get the result in both de Bruijn and classical notation:

$\lambda\lambda(2\ 1)$

$\lambda a. \lambda b. a \ b \Rightarrow 1$

Example 2: typing `S` at the prompt will yield the definition of combinator S : $\lambda\lambda\lambda((31)(21))$ in de Bruijn notation, or $\lambda a. \lambda b. \lambda c. (ac)(bc)$

Example 3: now assume you wanted to define the successor expression `SUCC`. You can do this by typing (mind the use of backslash for λ as well as the space after the dot):

```
mikrokosmos session
mikro> SUCC = \p. \q. \r. q (p q r)
```

Now evaluating `SUCC 0` should show the step-by-step reduction:

```
mikrokosmos session
mikro> SUCC 0

( $\lambda\lambda\lambda(2\ ((3\ 2)\ 1))\ \lambda\lambda 1$ )
 $\lambda\lambda(2\ ((\lambda\lambda 1\ 2)\ 1))$ 
 $\lambda\lambda(2\ (\lambda 1\ 1))$ 
 $\lambda\lambda(2\ 1)$ 

 $\lambda a. \lambda b. a \ b \Rightarrow 1$ 
```

So, the result is $\lambda a. \lambda b. a \ b$, which, as highlighted by the interpreter, is the definition of 1.

The **cyan** elements are the positions to be replaced in the next β -reduction, while the **dark green** ones are the values to be "plugged in".

Turning on SKI mode

If you also need the combinator-based version of a λ -expression, switch the `ski` mode on:

```
mikrokosmos session
mikro> :ski on
```

For `SUCC 0` in the example above, the interpreter will now reply $\lambda a. \lambda b. a \ b \Rightarrow I \Rightarrow 1$

The result is 1, which has the same representation as combinator I (you can check this by successively applying a β -reduction and an η -reduction on expression $1\ x$).

Loading files

You can store the definitions in a file and later load and use them; each expression must be defined before being used. To enter comments, start them with `#`. The name of the file storing the definitions always ends in `.mkr`, but this part must be omitted when loading it. For example, if the definition of `SUCC` is stored in file `foo.mkr`, one can load it by typing:

```
mikrokosmos session
mikro> :load foo
```

`mikrokosmos` seeks for `foo.mkr` in your home directory (`~` in short), in `~/.mikrokosmos`, in the current folder (aka `.`) and in `./lib`. Absolute and relative file names can also be used.

Inhibiting evaluation

To inhibit evaluation, one must use `!=` instead of `=` in an expression definition. Typically, when a definition is introduced, the expression to the right of the equality sign is simplified and then the result gets bound to the name at the left side of the equality sign. Sometimes however, performing simplification before parameter binding must be avoided. For example, when introducing the `Y` combinator, we should use `!=` instead of `=` to avoid an infinite loop, which is actually embedded into the definition of `Y`.

13.3 Practice problems



Note 13.3.1

Unless stated otherwise, you are expected to solve the problems by hand (i.e. without `mikrokosmos`).

Exercise 13.3.1

Given the following expressions in de Bruijn notation, write for each of them the form which uses variables:

1. $\lambda\lambda\ 1$
2. $\lambda\lambda(2\ 2)$
3. $\lambda\lambda\lambda((3\ 2)\ (3\ 1))$

Exercise 13.3.2

Given the definitions of S , K and I :

$$I := \lambda x.x$$

$$K := \lambda x.\lambda y.x$$

$$S := \lambda x.\lambda y.\lambda z.xz(yz)$$

simplify the following expressions which contain combinators, showing each step:

1. $S\ K\ K$
2. $K\ I\ S$
3. $S\ (K\ S)\ K$

Check the results in `mikrokosmos`.

Hint:

What evaluation strategy will you use: call-by-name or call-by-value? Is one better than the other?

Exercise 13.3.3

Given the definitions of $0, 1, 2, \dots$, **SUCC**, **PLUS**, combinators, do the following:

Subpoint 13.3.3.1

Follow, step by step, the demo provided by **mikrokosmos** for the equivalence:

SUCC 0 $\equiv$ 1

Subpoint 13.3.3.2

Follow, step by step, the demo provided by **mikrokosmos** for the equivalence:

PLUS 1 2 $\equiv$ 3

Subpoint 13.3.3.3

Find out the S-K representations for **0**, **1**, **2**

Subpoint 13.3.3.4

Simplify the expression **S K**

Subpoint 13.3.3.5

Check to see what the expression **3 SUCC 1** returns

Subpoint 13.3.3.6

Figure out what the following expression yields:

AA = \m. \n. \f. \x. n f (m f x)

Subpoint 13.3.3.7

Figure out what the following expression yields: **AM = \m. \n. \f. m (n f)**

Subpoint 13.3.3.8

Find what each of the following expressions yields:

- **2 3**
- **0 1**
- **1 0**
- **0 0**
- **0 1 2**
- **C**
- **C 0 1 2**

Optional subpoint 13.3.3.1

Can you write an expression **AP** having two parameters **x** and **y**, which returns x^y ?

Optional subpoint 13.3.3.2

Can you come up with an expression **SUB**, having two parameters **x** and **y**, which returns the difference $(x - y)$ by applying **x** times the predecessor expression over the second parameter **y**? Note: if **x** is lower than **y**, the result should be 0.

Optional subpoint 13.3.3.3

Consider the following expression: `ANAND = \m. \n. m n m`.

Show that `ANAND` behaves as `&` when applied on lambda expressions representing boolean values. Now try to find out what each of the following expressions yields:

- `ANAND 1 2`
- `ANAND 2 2`
- `ANAND 2 3`
- `ANAND 3 2`

Can you figure out what `ANAND a b` returns for `a` and `b` natural numbers in Church notation?

Optional subpoint 13.3.3.4

Can you suggest an expression `isOdd` which checks if its argument `n` is odd?

Exercise 13.3.4

Write an expression called `SQSUM` which adds up the squares of the first n natural numbers,

based on the recursive formula $\sum_{i=1}^n i^2 = n^2 + \sum_{i=1}^{n-1} i^2$.

Hint:

You may want to write the implementation in a file and test it by loading the file in the REPL, as explained above.